

ENTERPRISE SECURITY ASSESSMENT LAB

API Security Evidence Documentation Report

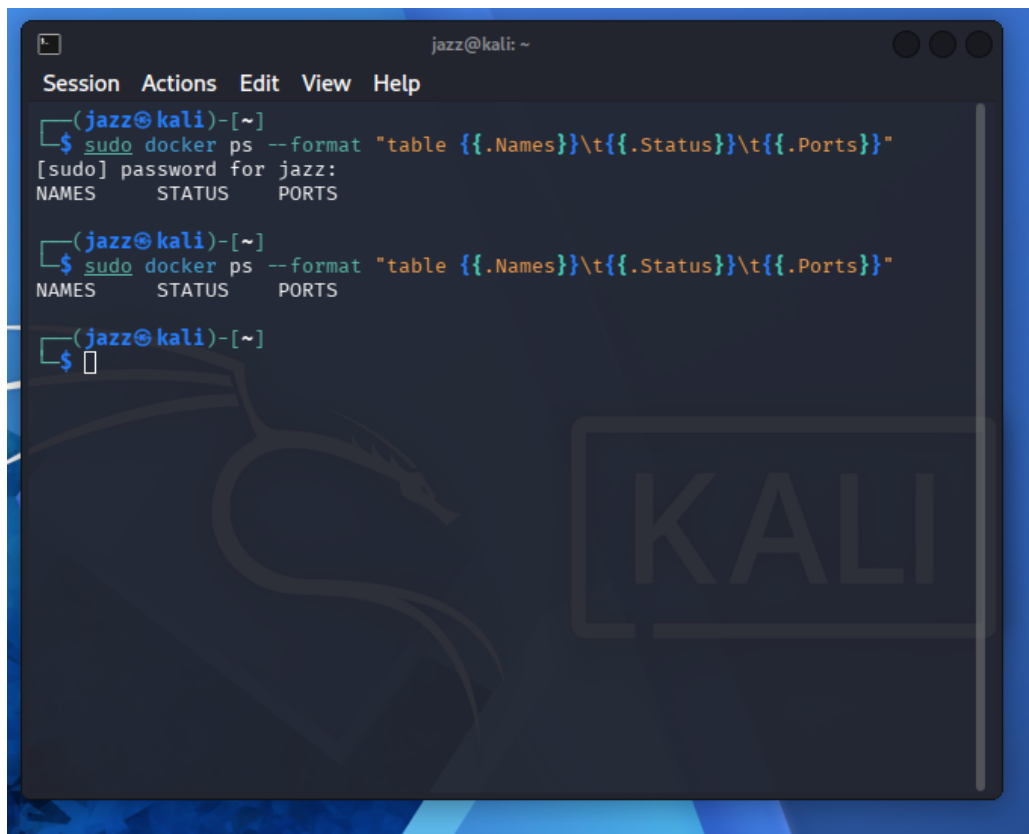
API Security Tool Readiness Evidence

Professional Portfolio Edition - Authorized API Security Lab Evidence

Field	Details
Test Area	API tools version check
Result Type	Tool readiness
Evidence Report File	05-api-tools-version-check-evidence-report.pdf
Testing Phase	Tool Verification
Tool / Component	Terminal / Python / API Testing Utilities
OWASP / Security Mapping	Assessment Preparation
Repository Path	02-api-security/reports/evidence/
Prepared For	GitHub recruiter portfolio documentation

This report documents one API Security evidence row as a standalone professional artifact. It includes the embedded screenshot, what it proves, the methodology context, command/workflow, sanitized output style, risk interpretation, remediation guidance, and publication redaction requirements.

1. Embedded Evidence Screenshot

A terminal window titled 'jazz@kali: ~' with a menu bar (Session, Actions, Edit, View, Help). The terminal shows three commands being entered: 1. '\$ sudo docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"' followed by '[sudo] password for jazz:' and the output 'NAMES STATUS PORTS'. 2. The same command is entered again, followed by the same output. 3. The command '\$' is entered, followed by a cursor. A large, faint 'KALI' watermark is visible in the background of the terminal window.

```
jazz@kali: ~
Session Actions Edit View Help
(jazz@kali)-[~]
$ sudo docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
[sudo] password for jazz:
NAMES      STATUS    PORTS

(jazz@kali)-[~]
$ sudo docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
NAMES      STATUS    PORTS

(jazz@kali)-[~]
$
```

Figure 1 - Embedded screenshot evidence for: API tools version check

What this evidence proves

The screenshot shows command-line checks for API testing tools. This confirms that the testing environment had the required capabilities before the API assessment began.

Evidence handling note

The screenshot has been prepared for GitHub publication. Sensitive fields such as credentials, tokens, JWTs, cookies, personal data, host values, or other secrets are blurred or represented as redacted values where needed.

2. Assessment Context and Methodology Placement

Area	Value
Testing Phase	Tool Verification
Tool / Component	Terminal / Python / API Testing Utilities
Assessment Objective	Verify that the required API security testing tools and libraries are installed before beginning the assessment.
Result Type	Tool readiness
OWASP / Security Mapping	Assessment Preparation

Why this evidence matters

Tool verification improves credibility and repeatability. It also gives context to later screenshots showing JWT analysis, ffuf, Kiterunner, Postman, and Burp workflows.

Command / workflow used

```
python3 --version
pip3 list | grep -E "jwt|requests"
ffuf -V
kr --version
curl --version
jq --version
```

Sanitized output style

```
Python / package checks complete
API tooling available
Environment ready for testing
```

3. Professional Interpretation

This evidence row should be interpreted as: **Tool readiness**.

This is not a vulnerability. It is readiness evidence. Tool versions can matter when reproducing results or troubleshooting differences.

Assessor notes

Useful supporting evidence before API test execution.

How a recruiter or reviewer should read this evidence

The value of this evidence is not only the screenshot itself. The value is that it demonstrates a structured API security testing process: controlled lab setup, targeted testing, safe interpretation of results, and clear separation between confirmed vulnerabilities and controls that rejected attack attempts.

Finding interpretation

Where the evidence shows vulnerable behaviour, the report should explain the security impact, the affected object or workflow, and the remediation without publishing sensitive raw values.

4. Risk, Impact, and Remediation Guidance

Risk perspective

This is not a vulnerability. It is readiness evidence. Tool versions can matter when reproducing results or troubleshooting differences.

Recommended remediation / hardening actions

- Keep tools updated.
- Record tool versions for repeatability.
- Avoid committing local package caches or secrets.
- Use a clean lab environment for portfolio work.

Validation guidance after remediation

- Repeat the same test workflow after remediation and compare the response behaviour.
- Confirm that sensitive data is no longer exposed in raw API responses.
- Confirm that unauthorized requests return secure error responses such as 401 or 403 where appropriate.
- Confirm that security controls are enforced server-side, not only in the frontend or client collection.

5. GitHub Publication and Redaction Guidance

Sensitive values to redact before public upload

- Local usernames if private.
- Paths containing personal names.
- Shell history containing secrets.

General API evidence redaction checklist

- Blur JWT access tokens, bearer tokens, OAuth codes, cookies, and authorization headers.
- Blur email addresses, phone numbers, user IDs, vehicle IDs, order IDs, and private object identifiers.
- Do not upload Postman environment files containing live variables or secrets.
- Do not upload raw response exports containing personal data or secret-bearing values.
- Use placeholders such as , , and .

GitHub link format for 02-api-security/README.md

[API tools version check](./reports/evidence/05-api-tools-version-check-evidence-report.pdf)

Final classification

This PDF is suitable for a public GitHub portfolio only after the embedded screenshot has been reviewed and sensitive values are confirmed blurred or removed.